CSE103: Structured Programming

Lecture 12

Dr. Sajeeb Saha, Dept. of CSE, JnU

Introduction

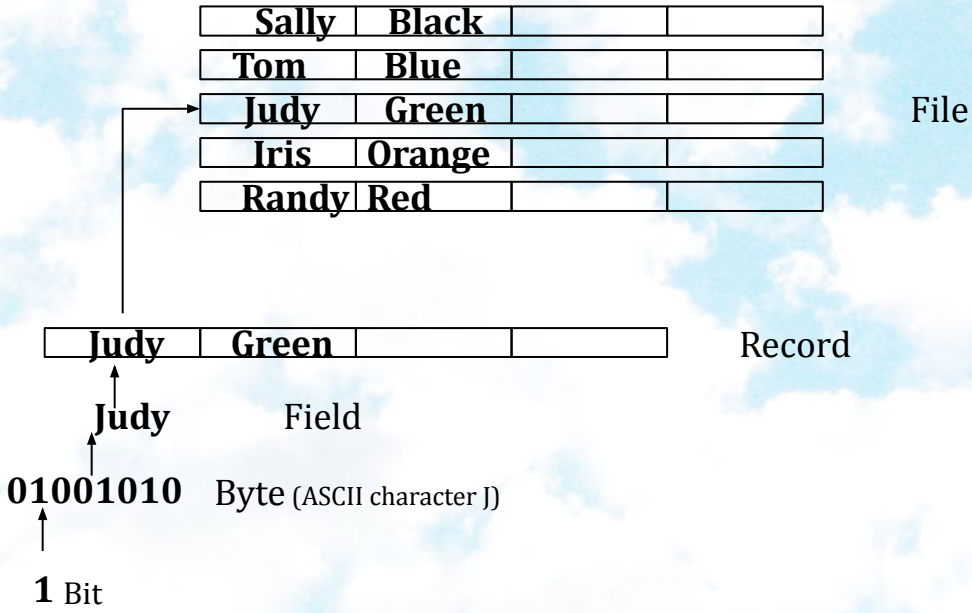
- Data files
  - Can be created, updated, and processed by C programs
  - Are used for permanent storage of large amounts of data
    - Storage of data in variables and arrays is only temporary

The Data Hierarchy

- Data Hierarchy:
  - Bit – smallest data item
    - Value of **0** or **1**
  - Byte – 8 bits
    - Used to store a character
      - Decimal digits, letters, and special symbols
  - Field – group of characters conveying meaning
    - Example: your name
  - Record – group of related fields
    - Represented by a **struct** or a **class**
    - Example: In a payroll system, a record for a particular employee that contained his/her identification number, name, address, etc.

The Data Hierarchy

- Data Hierarchy (continued):
  - File – group of related records
    - Example: payroll file
  - Database – group of related files



Files & Streams

- A file is an "independent entity" with a name recorded by the operating system.
- A stream is created by a program.
- To work with a file, we must associate our stream name with the file name recorded by the OS.

File operation Summary

- Create a file with a given name
- Open a file to use its contents
- Reading a file's content
- Updating a file's content
- Closing a file
- Delete a file

Steps in Processing a File

1. Create the stream via a pointer variable using the **FILE** structure: **FILE* spData;**
2. Open the file, associating the stream name with the file name.
3. Read or write the data.
4. Close the file.

File Open

- The file open function (**fopen**) serves two purposes:
  - It makes the connection between the physical file and the stream.
  - It creates "a program file structure to store the information" C needs to process the file.
- Syntax:

fopen("filename", "mode");

More On fopen

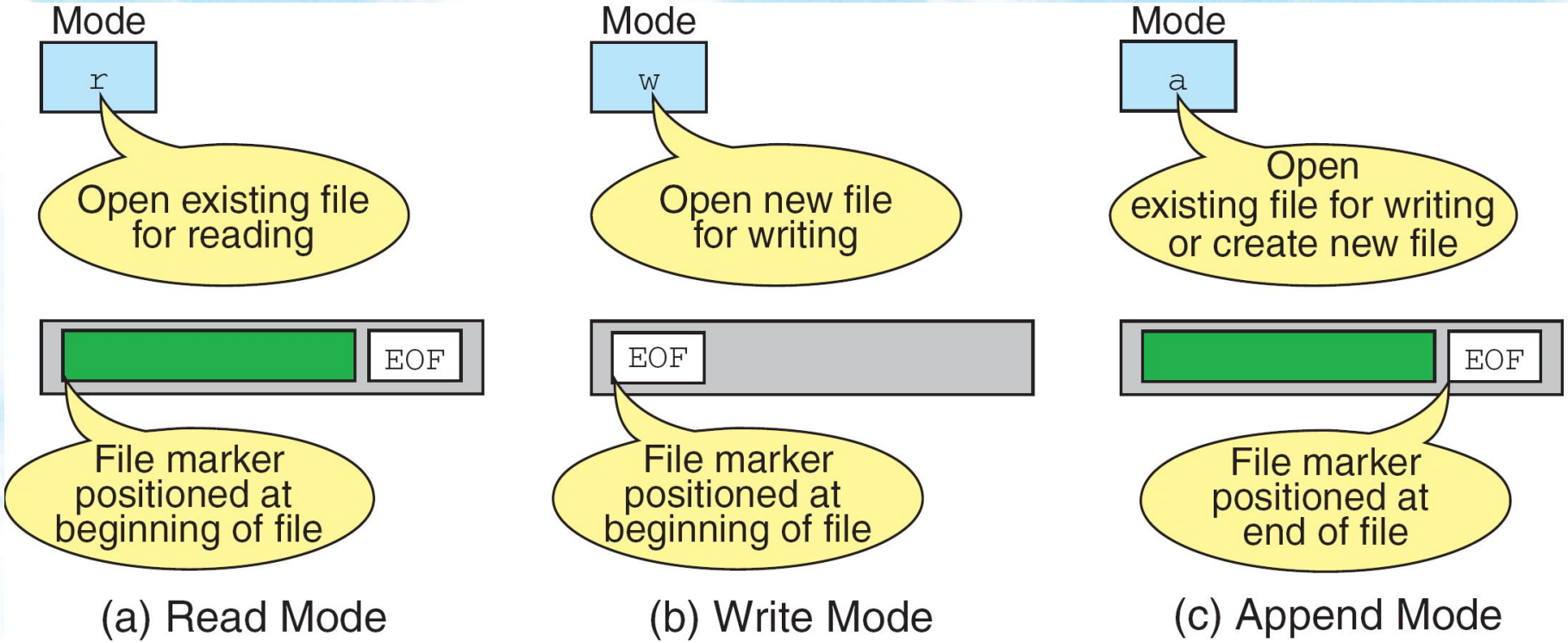
- The *file mode* tells C how the program will use the file.
- The *filename* indicates the system name and location for the file.
- We assign the return value of **fopen** to our pointer variable:

```
spData = fopen("MYFILE.txt", "w");
spData = fopen("A:\\MYFILE.txt", "w");
```

Opening a file using fopen

Mode			Description	Starts at position..
r	rb		open for reading	beginning
w	wb		open for writing (creates file if it doesn't exist). Deletes content and overwrites the file.	beginning
a	ab		open for appending (creates file if it doesn't exist)	end
r+	rb+	r+b	open for reading and writing	beginning
w+	wb+	w+b	open for reading and writing. Deletes content and overwrites the file.	beginning
a+	ab+	a+b	open for reading and writing (append if file exists)	end

More on File Open Modes



from Figure 7-4 in **Forouzan & Gilberg**, *p. 401*

Closing a File

- When we finish with a mode, we need to close the file before ending the program or beginning another mode with that same file.
- To close a file, we use **fclose** and the pointer variable:

```
fclose(spData);
```

Input/Output operations on files

- C provides several different functions for reading/writing
  - getc() – read a character
  - putc() – write a character
  - fprintf() – write set of data values
  - fscanf() – read set of data values
  - getw() – read integer
  - putw() – write integer

getc() and putc()

- handle one character at a time like getchar() and putchar()
- syntax: putc(c,fp1);
  - c : a character variable
  - fp1 : pointer to file opened with mode **w**
- syntax: c = getc(fp2);
  - c : a character variable
  - fp2 : pointer to file opened with mode **r**
- file pointer moves by one character position after every getc() and putc()
- getc() returns end-of-file marker EOF when file end reached

fscanf() and fprintf()

- similar to scanf() and printf()
- in addition provide file-pointer
- given the following
  - file-pointer f1 (points to file opened in write mode)
  - file-pointer f2 (points to file opened in read mode)
  - integer variable i
  - float variable f
- Example:

```
fprintf(f1, "%d %f\n", i, f);
fprintf(stdout, "%f \n", f);    /*note: stdout refers to screen */
fscanf(f2, "%d %f", &i, &f);
```

- fscanf returns EOF when end-of-file reached

Creating a file

```
#include<stdio.h>
int main()
{

   FILE *fp;
   fp=fopen("test13.txt","w");

   if(fp==NULL)
   {
      printf("File does not created!!!");
      exit(0); /*exit from program*/
   }

   printf("File created successfully.");

   int n;
   printf("\n Enter a number:");
   scanf("%d",&n);

   fprintf(fp,"%d",n);
   fclose(fp);

   return 0;
}
```

Reading a file

```
#include<stdio.h>
int main()
{

   FILE *fp;
   fp=fopen("test16.txt","r");

   if(fp==NULL)
   {
      printf("File does not exist!!!");
      exit(0); /*exit from program*/
   }

   int num;
   fscanf(fp,"%d", &num);
   printf("The value is : %d",num);

   fclose(fp);

   return 0;
}
```

THANK
YOU!

QUESTIONS?
COMMENTS?
CONCERNS?